LiquidClips: Technical Architecture Overview

Built by Emmanuel Eseyin, full-stack developer. Live at liquidclips.app

The big picture

LiquidClips is a layered product, not a single-stack app. It is made of three specialized parts:

- **Frontend shell:** React, TypeScript and Tauri
- **Media engine:** a Python sidecar
- **Authority layer:** a FastAPI backend

Each layer has a different job. The interface handles interaction, the media engine handles local processing, and the backend is the trusted source of truth for authentication, billing, permissions and business data.

What each layer owns

- **Frontend shell owns the experience.** It handles routes, onboarding, account flows, local workflows and everything the user sees. React suits an interactive interface, TypeScript reduces bugs in a complex app, and Tauri provides a native desktop shell while keeping the interface web-based.
- **Python sidecar owns processing.** It handles transcription, ffmpeg transforms, clip generation, reframing, thumbnails and AI judgment. Heavy media work does not belong in the browser layer, so it runs locally in its own process.
- **FastAPI backend owns truth.** It handles authentication, JWT validation, user profile hydration, subscriptions, permissions, billing state and admin data. It decides what the system can trust.

Each layer stays inside its lane.

Why not a single app

A single monolithic app would create three problems:

- Interface and media processing have different performance needs
- Browser-based media work is unreliable for heavy local processing

- Identity, billing and permissions need central trust and enforcement

So the system is deliberately separated into specialized layers.

State management

State is split by purpose instead of being forced into one giant store:

- **Local component state:** temporary interface state such as modals, form values and toggles. Cheap and easy to reason about.
- **Zustand stores:** domain-based app state such as campaigns, community, rewards, engine session and user profile, giving app-wide reactivity without heavy prop drilling.
- **Event-driven coordination:** workflow and navigation patterns coordinated through events, which avoids tight coupling and keeps route transitions consistent.
- **Server-backed identity state:** user identity is hydrated from the backend, because access and tier checks must be trusted.
- **Local persistence:** tokens, route state, settings and session memory are stored locally for continuity, but never treated as the authority for sensitive logic.

How operations flow

1. **Start-up:** the shell loads and checks auth state. It either hydrates the user's identity or routes into onboarding.
2. **Action:** the user triggers a workflow, local state updates, and the app calls the backend or starts a sidecar process when needed.
3. **Media processing:** the sidecar runs the heavy local pipeline and reports progress back to the interface.
4. **Backend operations:** identity, billing, permissions and platform actions go through the FastAPI backend.

Why this architecture works

- The interface stays fast and interactive
- Media processing is local and can be heavy without slowing the UI
- Authentication and business logic are centralized and secure
- State is organized by purpose, so the app stays maintainable as it grows

Summary

LiquidClips is built as a three-layer system. The desktop app uses React and Tauri for the user-facing shell. The Python sidecar handles heavy media processing like transcription and clip generation. The FastAPI backend is the source of truth for authentication, billing, permissions and platform data.